Numerical Investigation of Flow Past a Circular Cylinder using OpenFOAM

Martin Mathew

Faculty of Maritime Studies
University of Split
Split, Croatia

Supervisor: Prof. Andro Bakica

Declaration of Originality

I hereby declare that this report is my own original work. The underlying research, the design of the parametric meshing and automation framework, and the analysis and conclusions presented herein are my own. All sources consulted during this investigation have been properly cited in accordance with academic conventions, and no portion of this work has been submitted previously for assessment at this or any other institution.

In the preparation of this report, an AI-based language model (Anthropic's Claude) was used as a writing and coding assistance tool, specifically to help structure and phrase sections of the written report, and to support the optimisation, debugging, and documentation of the supporting OpenFOAM configuration files and shell/Python scripts developed for this project.

Signature: _____ Date: September 4, 2026

Abstract

Computational studies of flow past a circular cylinder are frequently limited, in practice, not by solver capability but by the effort required to generate a consistent, high-quality mesh across the range of Reynolds numbers and resolutions needed for a credible parametric campaign. This project addresses that limitation by developing a fully parameterised, block-structured meshing and case-automation framework for incompressible flow past a circular cylinder in OpenFOAM, in which every geometric and grading quantity in the mesh is derived analytically, via OpenFOAM's `#calc` macro, from a small set of independent control parameters.

The mesh itself adopts a nested double O-grid topology: an inner ring of hexahedral blocks dedicated to resolving the near-wall viscous region, and an outer ring that buffers the transition into a far-field H-grid, with the far-field cell counts and gradings derived from a matched interface cell width so that cell size remains continuous across every internal block boundary. This mesh-generation logic is exercised at scale by a dedicated driver script that instantiates, configures, meshes, and solves an arbitrary number of cases from a single tabulated specification, applying a template-copy instantiation pattern and per-case, per-stage fault isolation suited to unattended execution of large parametric sweeps.

Building on this infrastructure, the project further specifies a verification and validation protocol, covering mesh independence, temporal convergence, domain-size sensitivity, and validation against published benchmark data, and shows that each of these studies reduces, within the present framework, to varying one or two parameters of the case specification rather than requiring custom manual intervention.

Contents

Declaration of Originality iii

List of Tables v

1 Introduction 1
- 1.1 Motivation and Background . . . 1
- 1.2 Research Objectives and Report Structure . . . 1

2 Background 3
- 2.1 Classical Experimental Studies . . . 3
- 2.2 Reynolds Number Regime Classification . . . 3
- 2.3 Computational Studies of Laminar Cylinder Flow . . . 4
- 2.4 RANS Modelling of Turbulent Cylinder Flow . . . 4
- 2.5 Large Eddy Simulation of Cylinder Wakes . . . 5
- 2.6 Data-Driven and AI-Assisted CFD . . . 5
- 2.7 Governing Equations for Incompressible Flow . . . 6
- 2.8 Reynolds Decomposition and the RANS Equations . . . 6
- 2.9 Force Coefficients and Strouhal Number . . . 7

3 Computational Methodology 9
- 3.1 Overview . . . 9
- 3.2 Physical Geometry and Domain Dimensions . . . 10
- 3.3 Parametric Double O-Grid Topology . . . 12
- 3.4 Mesh Grading Strategy . . . 12
- 3.5 Boundary Conditions . . . 13
- 3.6 Numerical Schemes and Solver Configuration . . . 14
- 3.7 Reference Quantities . . . 15
- 3.8 Automated Parametric Campaign Execution . . . 16

4 Automated Simulation Campaign Framework 17
- 4.1 Motivation for Campaign-Level Automation . . . 17
- 4.2 Repository and Directory Architecture . . . 17
- 4.3 Case Specification Schema . . . 19
- 4.4 Execution Pipeline and Fault Tolerance . . . 20
- 4.5 Logging and Campaign Traceability . . . 21

5 Verification and Validation: Methodological Potential 23
- 5.1 Overview . . . 23

5.2 Mesh, Time-Step, and Reynolds-Number Sensitivity 23

6 Conclusions and Future Work **25**
6.1 Summary of the Work 25
6.2 Key Contributions 26
6.3 Limitations 26
6.4 Future Work 27
6.5 Closing Remarks 27

Bibliography **29**

A Appendix **I**
A.1 Case Cleanup Script: `cleanCases.sh` I
A.2 Case Execution Script: `runCases.sh` III
A.3 Post-Processing Script: `postProcess.py` VII
A.4 ParaView File Generation Script: `touch_foam.sh` VIII

List of Tables

3.1 Computational domain dimensions, non-dimensionalised by the cylinder diameter D, for the baseline configuration ($D = 0.1$ m). 10
3.2 Parametric variables defined in `geometryProperties` and referenced throughout `blockMeshDict` for the baseline configuration. 11
3.3 Mesh grading parameters, their governing expressions, and their evaluated values for the baseline configuration ($E = 1.5$, $N_{bl} = 10$, $E_{far} = 1.06$, $N_c = 60$). 13
3.4 Boundary patches and geometric types defined in `blockMeshDict`. 14
3.5 Case-dependent solver and time-integration parameters, as substituted into `system/controlDict` and `system/fvSchemes` by `runCases.sh`. 15

4.1 Column schema of `cases.csv` and the file or entry configured by each column. 19
4.2 Sample campaign rows from `cases.csv`. 20

5.1 Parameters varied for each potential V&V study within the existing campaign framework. 24

1
Introduction

1.1 Motivation and Background

The study of flow around bluff bodies represents a foundational pillar of applied fluid mechanics. Unlike streamlined bodies, bluff geometries generate separated wake regions whose unsteady dynamics strongly influence the forces experienced by structures across a vast range of engineering applications. Among the many canonical bluff-body configurations, the circular cylinder has attracted particular and sustained scientific interest for more than a century, primarily because of the exceptional clarity with which it exhibits the essential phenomena of boundary-layer separation, periodic vortex formation, and pressure-induced drag.

In practical engineering, cylindrical or near-cylindrical cross-sections appear in bridge piers, offshore risers, heat-exchanger tube bundles, overhead transmission lines, chimney stacks, and marine mooring cables. In each of these contexts, the alternating pressure loading associated with periodic wake shedding can drive resonant structural vibrations whose amplitude grows dramatically when the shedding frequency couples with a natural frequency of the structure—a mechanism known as lock-in or vortex-induced vibration (VIV). The consequences range from fatigue damage and reduced service life to catastrophic structural failure, as evidenced by the 1940 collapse of the Tacoma Narrows Bridge [6]. Predictive computational models that reliably capture shedding frequency, mean drag, and fluctuating lift are therefore indispensable design tools.

1.2 Research Objectives and Report Structure

The overarching objective of this investigation is to develop a progressively refined, multi-fidelity computational framework for the analysis of incompressible flow past a circular cylinder. Central to this objective is the creation of a fully automated, heavily parameterized OpenFOAM case structure. By driving the domain geometry, mesh topology, and solver execution entirely through an automated pipeline, this study aims to systematically escalate modelling complexity while maintaining mathematical consistency across all fidelity levels. This approach bridges the gap between traditional computational fluid dynamics (CFD) and data-driven surrogate modelling.

To realise this overarching goal, the specific objectives are defined as follows:

1. **Development of a Parametric Meshing Architecture:** Construct a robust, automated meshing utility using a bespoke `geometryProperties` dictionary. This objective involves formulating mathematically linked relationships that control near- wall boundary layer clustering, double O-grid transition smoothing, and far- field spatial discretization, allowing rapid iteration across diverse flow regimes without manual grid regeneration.

2. **Baseline Laminar Configuration and Validation:** Establish a rigorously verified laminar solver configuration within the automated environment. This foundational step will validate mean and unsteady aerodynamic force coefficients, Strouhal numbers, and wake topologies against established theoretical and experimental benchmarks from the open literature.

3. **Development of a 3D Large Eddy Simulation (LES) Framework:** Formulate and validate an unsteady LES framework on three-dimensional computational domains. The automated script will be utilized to extrude the domain spanwise and heavily refine the wake region, focusing on the sensitivity of predicted flow statistics and coherent structures to the choice of subgrid-scale (SGS) models.

Structure of the Report

The remainder of this report is organised as follows to systematically address the aforementioned objectives.

Chapter 2 presents a focused review of the pertinent literature, establishing the current state-of-the-art in cylinder wake dynamics, multi-fidelity CFD, and machine learning applications in fluid mechanics.

Chapter 3 outlines the computational methodology, providing an in-depth discussion of the domain geometry, meshing strategy, boundary conditions, and the spatial and temporal numerical discretization schemes employed in OpenFOAM.

Chapter 4 describes the campaign-automation framework built around this meshing strategy, detailing the directory architecture, the tabulated case-specification schema, the case-instantiation and parameter-injection pipeline, and the fault-tolerant execution and logging procedures used to generate and run an arbitrary number of parametrically distinct cases without manual intervention.

Chapter 5 sets out the verification and validation methodology by which the accuracy and reliability of the resulting simulations are to be established, outlining how mesh-independence, temporal-convergence, and Reynolds-number sweep studies are conducted within the existing framework by varying a small number of case parameters.

Finally, Chapter 6 synthesises the key findings and contributions of the work, discusses the limitations of the present framework, and identifies directions for future development and extension of the methodology.

2

Background

This chapter presents the foundational concepts necessary for understanding the scope and significance of this research. It introduces the operational principles of fixed-bed reactors, their applications in industry, underlying transport phenomena, and key computational tools relevant to their modeling.

2.1 Classical Experimental Studies

The scientific exploration of circular-cylinder wakes can be traced to the systematic flow visualisation experiments of von Kármán and Rubach in the early twentieth century, who provided the first theoretical account of the stability characteristics of the staggered double vortex row now bearing von Kármán's name [20]. Roshko's landmark experimental work [38] established the empirical relationship between Strouhal number and Reynolds number across a wide parameter range and introduced the concept of a universal wake Strouhal number based on wake width rather than cylinder diameter.

Williamson's contributions [47, 49] refined understanding of the transitions between distinct wake regimes, documenting the discontinuous jumps in Strouhal number associated with changes in vortex-filament topology (the so-called A- and B-mode three-dimensional instabilities), and providing meticulously measured force coefficient and frequency data that have since become the primary validation reference for computational studies. Zdravkovich's two-volume compilation [51] surveys the full body of experimental knowledge on cylinder aerodynamics and remains an indispensable reference.

Achenbach [1] investigated the high-Reynolds-number regime in detail, documenting the drag crisis and its sensitivity to surface roughness and free-stream turbulence intensity. More recently, quantitative techniques such as particle-image velocimetry (PIV) have been applied to cylinder wakes to provide spatially resolved velocity and vorticity fields suitable for direct comparison with simulation outputs [32].

2.2 Reynolds Number Regime Classification

The recognised classification of cylinder wake regimes provides the organisational framework for the present investigation. Following [51] and [49], the principal regimes

may be summarised as follows:

- **Creeping / Stokes flow** ($Re < 1$): Flow is fully attached and reversible; viscous forces dominate.
- **Steady separated wake** ($1 < Re < 47$): A recirculating bubble forms; the flow remains steady and symmetric.
- **Laminar periodic shedding** ($47 < Re < 190$): Periodic two-dimensional shedding; well-defined Strouhal number.
- **Three-dimensional transition** ($190 < Re < 1000$): Spanwise instabilities (modes A and B) introduce three-dimensional structure.
- **Irregular turbulent wake** ($Re > 10^3$): Increasingly disordered and broadband shedding.
- **Drag crisis** ($Re \approx 3 \times 10^5$–5×10^5): Boundary-layer transition, delayed separation, and sharp drag reduction.

2.3 Computational Studies of Laminar Cylinder Flow

Two-dimensional laminar simulations of cylinder flow have been reported by numerous investigators. Posdziech and Grundmann [34] employed a spectral- element method on extremely large computational domains to provide resolution-independent reference values of $\overline{C}_D$, $C_{L,\mathrm{rms}}$, and St for $Re = 20$–250, accounting carefully for blockage effects. Their data have been widely adopted as reference values for code-verification exercises. Barkley and Henderson [3] used Floquet stability analysis to characterise the three-dimensional wake instabilities, establishing accurate threshold Reynolds numbers for mode A and mode B onset. Liu et al. [25] performed detailed time-dependent simulations confirming the accuracy of the von Kármán shedding mechanism and reporting pressure distributions that compare favourably with experiment.

Finite-volume-based studies using OpenFOAM or comparable tools have reproduced these benchmark results with second-order schemes on structured and block-structured meshes [37, 12], confirming that the platform is capable of accurate laminar predictions when mesh resolution and domain extent are chosen appropriately.

2.4 RANS Modelling of Turbulent Cylinder Flow

RANS simulations of cylinder flow in the turbulent regime present well-known difficulties. Franke and Rodi [17] demonstrated that standard linear eddy-viscosity models tend to overpredict eddy viscosity in the separating shear layers, suppress the shedding oscillation, and misrepresent the recirculation length. The two-layer models of that era showed modest improvements near the surface but poor wake recovery. Catalano et al. [10] compared several RANS models with LES reference data at $Re = 10^6$ and found substantial model-to-model scatter, with all RANS closures underestimating base pressure suction and over-predicting recirculation length to varying degrees.

The k–ω SST model of [27] has shown superior performance in separated and adverse-pressure-gradient flows compared with k–ε variants, owing to its selective use of the ω-equation in near-wall regions and the ε-formulation in the far field. Nevertheless, even this model encounters difficulties with the highly curved streamlines in the near wake of a cylinder.

2.5 Large Eddy Simulation of Cylinder Wakes

LES approaches have demonstrated considerably higher predictive fidelity for cylinder wakes. The pioneering LES study of [4] established that LES with the Smagorinsky subgrid model could reproduce experimental velocity spectra and force statistics at $Re = 3900$ provided the mesh was sufficiently resolved in the azimuthal direction. Subsequent investigations at the same Reynolds number, including those by [16] and [32], have established a detailed dataset against which new subgrid models and numerical schemes can be assessed. Breuer [7, 8] conducted extensive parametric LES studies over a wide Reynolds number range, identifying the sensitivity of predicted recirculation length and base pressure to subgrid model, mesh resolution, and spanwise domain extent.

The dynamic version of the Smagorinsky model, first proposed by [18] and reformulated by [23], removes the necessity of prescribing the Smagorinsky constant by locally and dynamically computing it from the resolved velocity field. The WALE (Wall-Adapting Local Eddy-viscosity) model [28] additionally vanishes naturally at solid walls without explicit damping functions, offering practical advantages for wall-resolved cylinder simulations.

2.6 Data-Driven and AI-Assisted CFD

The integration of machine learning with computational fluid dynamics has accelerated rapidly over the past decade. Supervised neural network architectures have been trained on CFD datasets to construct surrogate models capable of predicting force coefficients and wake statistics at a fraction of the cost of full simulations [9, 22]. Physics-Informed Neural Networks (PINNs), introduced by [35], embed the governing partial differential equations as soft constraints in the loss function, enabling data-sparse reconstruction of flow fields and identification of physical parameters. Their application to cylinder-wake reconstruction from sparse sensor measurements has demonstrated the potential of the approach for real-time flow estimation [19].

Proper Orthogonal Decomposition [26, 42] extracts a low-dimensional basis from an ensemble of flow snapshots, capturing the dominant coherent structures—including the vortex-shedding mode—in the leading modes. The resulting reduced-order model (ROM) can reproduce the unsteady dynamics at orders-of-magnitude lower cost than the full simulation, making it attractive for real-time control and optimisation applications [29, 43]. The multi-fidelity dataset generated by the present investigation provides an ideal resource for exploring these techniques systematically.

2.7 Governing Equations for Incompressible Flow

The motion of an incompressible, Newtonian fluid is governed by the continuity equation expressing conservation of mass,

$$\nabla \cdot \mathbf{u} = 0, \tag{2.1}$$

and the Navier–Stokes momentum equations,

$$\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla)\mathbf{u} = -\frac{1}{\rho}\nabla p + \nu \nabla^2 \mathbf{u}, \tag{2.2}$$

where $\mathbf{u}(\mathbf{x}, t)$ is the velocity field, $p(\mathbf{x}, t)$ the kinematic pressure (pressure divided by density), ρ the (constant) fluid density, and $\nu = \mu/\rho$ the kinematic viscosity. In the current study, air at standard conditions is the working fluid, with $\nu = 1.5 \times 10^{-5}\,\mathrm{m^2\,s^{-1}}$. The cylinder diameter is $D = 0.1\,\mathrm{m}$, and the free-stream velocity for a desired Reynolds number follows from re-arranging Reynolds equation,

$$U_\infty = \frac{Re \cdot \nu}{D}. \tag{2.3}$$

2.8 Reynolds Decomposition and the RANS Equations

In turbulent flow, each flow variable ϕ is decomposed into a time-averaged component $\overline{\phi}$ and a fluctuating component ϕ', such that $\phi = \overline{\phi} + \phi'$ and $\overline{\phi'} = 0$. Applying this Reynolds decomposition to Equations (2.1) and (2.2) and time-averaging yields the Reynolds-Averaged Navier–Stokes (RANS) equations:

$$\nabla \cdot \overline{\mathbf{u}} = 0, \tag{2.4}$$

$$\frac{\partial \overline{\mathbf{u}}}{\partial t} + (\overline{\mathbf{u}} \cdot \nabla)\overline{\mathbf{u}} = -\nabla \overline{p} + \nu \nabla^2 \overline{\mathbf{u}} - \nabla \cdot \boldsymbol{\tau}^R, \tag{2.5}$$

where the Reynolds stress tensor is defined as

$$\tau_{ij}^R = \overline{u_i' u_j'}. \tag{2.6}$$

The Reynolds stress tensor introduces six additional unknowns into a system of four equations, constituting the well-known *closure problem*. Linear eddy-viscosity models resolve this by invoking the Boussinesq hypothesis,

$$\tau_{ij}^R = -\nu_t \left(\frac{\partial \overline{u}_i}{\partial x_j} + \frac{\partial \overline{u}_j}{\partial x_i} \right) + \frac{2}{3} k \delta_{ij}, \tag{2.7}$$

where ν_t is the turbulent (eddy) viscosity and $k = \frac{1}{2}\overline{u_i' u_i'}$ is the turbulent kinetic energy.

2.9 Force Coefficients and Strouhal Number

The macroscopic aerodynamic forces acting on the cylinder surface, which arise from the integration of pressure fields and viscous shear stresses, are conventionally expressed in non-dimensional form. The instantaneous drag and lift coefficients, denoted as C_D and C_L respectively, are defined as:

$$C_D = \frac{F_D}{\frac{1}{2}\rho U_\infty^2 DL}, \qquad C_L = \frac{F_L}{\frac{1}{2}\rho U_\infty^2 DL}, \tag{2.8}$$

where F_D and F_L represent the total streamwise (drag) and cross-stream (lift) forces. The denominator term $\frac{1}{2}\rho U_\infty^2$ is the free-stream dynamic pressure, D is the cylinder diameter, and L defines the spanwise length of the computational domain (taken as unity in two-dimensional simulations).

Furthermore, the unsteady, oscillatory dynamics of the cylinder wake, characterized by the alternating separation of von Kármán vortices, are quantified using the dimensionless Strouhal number (St). This parameter relates the dominant vortex shedding frequency to the macroscopic length and velocity scales of the flow:

$$St = \frac{f_s D}{U_\infty}, \tag{2.9}$$

where f_s is the primary shedding frequency in Hertz. In this study, f_s is extracted by performing a Fast Fourier Transform (FFT) on the fluctuating lift coefficient signal over a statistically stationary time window.

3

Computational Methodology

3.1 Overview

The computational methodology adopted in this study is organised around four principal components: (i) definition of the physical geometry and the computational domain, (ii) parametric mesh generation using OpenFOAM's native `blockMesh` utility, (iii) specification of boundary conditions, and (iv) configuration of the numerical schemes and solver settings. Because the circular cylinder is a geometrically simple body, the entire mesh is constructed analytically within a single `blockMeshDict` file without recourse to external CAD software. This approach guarantees full reproducibility and allows all geometric and mesh-density parameters to be varied through a single, centrally managed `geometryProperties` dictionary, whose entries are evaluated at run time via OpenFOAM's `\#calc` macro.

Unlike a single-block or single-O-grid discretisation, the near-cylinder region is resolved using a *nested double O-grid* topology: an inner ring of four hexahedral blocks dedicated to the viscous sublayer, and an outer ring of four hexahedral blocks acting as a graded buffer before the mesh transitions into an eight-block far-field H-grid (Section 3.3). Furthermore, unlike a strictly two-dimensional formulation obtained through an `empty` front/back patch, the present mesh is genuinely three-dimensional: the `frontAndBack` patch is assigned the `wall` type and the spanwise direction is resolved with $N_z = 10$ cells, so that spanwise flow structures can, in principle, be captured rather than suppressed by construction. Case-to-case variation of both the physical parameters (Reynolds number, solver, time-integration scheme) and the mesh-density parameters is orchestrated externally by the `runCases.sh` driver script operating on a tabulated case list, `cases.csv` (Section 3.8), rather than by manual editing of individual case dictionaries.

For more complex bodies, bluff shapes with surface curvature discontinuities, sharp trailing edges, or three-dimensional features, this strategy would be extended by first generating a surface model in a CAD environment and subsequently using `snappyHexMesh` to cut and refine a background mesh around the imported geometry; for the circular cylinder considered here, the fully analytic block-structured approach is preferred for the exactness and reproducibility it affords.

3.2 Physical Geometry and Domain Dimensions

The cylinder diameter D is the single master length scale of the mesh; all other geometric quantities are expressed in `geometryProperties` as exact multiples of D, so that a single change to D propagates automatically and consistently to every block vertex, arc definition, and domain boundary. For the baseline configuration examined in this chapter, $D = 0.1\,\text{m}$ and $R = D/2 = 0.05\,\text{m}$. The domain extents are summarised in Table 3.1.

Table 3.1: Computational domain dimensions, non-dimensionalised by the cylinder diameter D, for the baseline configuration ($D = 0.1\,\text{m}$).

Parameter	Symbol	Expression	Value (m)
Upstream length	X_{min}	$-10D$	-1.000
Downstream length	X_{max}	$+20D$	$+2.000$
Cross-stream lower bound	Y_{min}	$-10D$	-1.000
Cross-stream upper bound	Y_{max}	$+10D$	$+1.000$
Spanwise extent (front)	Z_{min}	$-0.5D$	-0.050
Spanwise extent (back)	Z_{max}	$+0.5D$	$+0.050$

The cross-stream boundaries at $Y_{min} = -10D$ and $Y_{max} = +10D$ yield a domain blockage ratio of

$$\beta = \frac{D}{Y_{max} - Y_{min}} = \frac{D}{20D} = 5.0\,\%. \tag{3.1}$$

This value sits at the upper edge of the threshold ($\beta < 5\,\%$) commonly regarded as sufficient to neglect blockage-correction effects when comparing against unconfined experimental or reference numerical data; it is not comfortably below this threshold, and a domain-sensitivity check (e.g. repeating a baseline case with Y_{max}, Y_{min} scaled outward) is advisable before the blockage-free assumption is relied upon quantitatively in the results chapters.

The independent and derived geometric parameters defined in `geometryProperties` and referenced throughout `blockMeshDict` are summarised in Table 3.2.

Table 3.2: Parametric variables defined in `geometryProperties` and referenced throughout `blockMeshDict` for the baseline configuration.

Variable	Definition	Value	Role
`D`	input	0.1 m	Cylinder diameter
`R`	D/2	0.05 m	Cylinder radius
`R_prism`	D*R_prism_mult	0.06 m	Inner O-grid (prism) boundary radius
`R_sq`	1.5*D	0.15 m	Outer O-grid boundary radius
`X_min`	-10*D	-1.0 m	Inlet face x-coordinate
`X_max`	20*D	$+2.0$ m	Outlet face x-coordinate
`Y_min`	-10*D	-1.0 m	Bottom face y-coordinate
`Y_max`	10*D	$+1.0$ m	Top face y-coordinate
`Z_min`	-0.5*D	-0.05 m	Front face z-coordinate
`Z_max`	0.5*D	$+0.05$ m	Back face z-coordinate
`ds_outer`	3*D/N_c	5.0×10^{-3} m	Matched cell width at O-grid/H-grid interface
`C`	$1/\sqrt{2}$	0.7071	Diagonal projection constant
`R_C`	D*0.5*C	3.536×10^{-2} m	Cylinder-block diagonal vertex offset
`R_prism_C`	D*R_prism_mult*C	4.243×10^{-2} m	Prism-block diagonal vertex offset
`l_Ref`	D	0.1 m	Reference length for Re and St
`A_Ref`	1.0*D*D	1.0×10^{-2} m^2	Reference area for C_D and C_L

The diagonal projection constant $\texttt{C} = 1/\sqrt{2} \approx 0.7071$ gives the Cartesian coordinate (equal in x and y) of the four 45°-diagonal vertices bounding both the cylinder-wall block (`R_C`) and the prism block (`R_prism_C`), ensuring that the circular arc edges can be specified with a single mid-arc point per edge and that the resulting hexahedral faces remain geometrically regular at the block corners.

3.3 Parametric Double O-Grid Topology

In place of a single H-grid block deformed to conform to the cylindrical wall, which produces severely skewed, high-aspect-ratio cells at the four points where a rectangular block boundary is tangent to a circular surface, the near-body mesh is constructed from two concentric rings of four hexahedral blocks each, prior to transitioning into the outer H-grid far field. This is evident directly in the `blocks` entry of `blockMeshDict`:

1. **Inner O-grid (prism layer):** four blocks connecting the cylinder wall at R to the prism boundary at R_{prism}, resolved with $N_{bl} = 10$ cells in the radial direction and $N_c = 60$ cells circumferentially.
2. **Outer O-grid (transition layer):** four blocks connecting R_{prism} to the O-grid outer square at $R_{sq} = 1.5D$, resolved with N_{trans} cells radially.
3. **Outer H-grid (far field):** eight blocks connecting the O-grid outer square to the rectangular far-field boundary.

Because each block in the two O-grid rings is a topologically regular quadrilateral rather than a wedge-shaped sector, cell faces in the near-wall region remain close to orthogonal to the wall normal across the full circumference, avoiding the corner singularity characteristic of a single-block polar discretisation. The outer ring functions specifically as a buffer between the aggressively graded inner prism layer and the comparatively coarse far-field H-grid: without it, a single block would be required to accommodate the full expansion from the near-wall cell size to the far-field cell size, producing poor orthogonality and high skewness at the O-grid/H-grid corners. By interposing a transition block with its own independently specified expansion rate (E_{far}; Section 3.4), the total cell-size growth is distributed across two grading stages, and cell aspect ratio remains bounded throughout the near-body region.

The cylindrical wall and the prism-ring boundary are both defined as true circular arcs (OpenFOAM `arc` edge type with a specified mid-arc point), rather than being approximated by straight chords, so that the discretised wall geometry converges to the exact circular boundary as N_c is increased.

3.4 Mesh Grading Strategy

All blocks employ `simpleGrading`, which prescribes a constant cell expansion ratio along each local block direction. Two independent expansion rates are specified by the analyst—the near-wall growth rate E and the target far-field expansion ratio E_{far}—from which four derived grading scalars are computed and reused throughout `blockMeshDict` to ensure consistent transitions at every shared block interface. For a geometric sequence of N cells with constant ratio r between successive cell widths, OpenFOAM's `simpleGrading` parameter is the ratio of the final to the first cell width, r^{N-1}. The four derived gradings are summarised in Table 3.3.

Table 3.3: Mesh grading parameters, their governing expressions, and their evaluated values for the baseline configuration ($E = 1.5$, $N_{bl} = 10$, $E_{far} = 1.06$, $N_c = 60$).

Parameter	Expression	Value	Physical role
`BL_grading`	$E^{N_{bl}-1}$	38.44	Total expansion across the inner prism layer, directing cells to grow away from the wall.
`Trans_grading`	$E_{far}^{N_{trans}-1}$	3.82	Total expansion across the outer transition layer.
`G_xMax`, `G_yMax`	E_{far}^{N-1}	21.94, 10.29	Applied in far-field blocks whose local block coordinate increases in the direction of growth (downstream, outward-lateral).
`G_xMin`, `G_yMin`	$E_{far}^{-(N-1)}$	0.0770, 0.0972	Applied in far-field blocks whose local block coordinate increases *toward* the cylinder, inverting the grading so the physical cell size still decreases toward the O-grid interface.

3.5 Boundary Conditions

Five named boundary patches are defined in `blockMeshDict`; their geometric type is summarised in Table 3.4.

Table 3.4: Boundary patches and geometric types defined in `blockMeshDict`.

Patch	Mesh type	Notes
`cylinder`	`wall`	Composed of four faces (one per inner O-grid block) at each of Z_{min} and Z_{max}.
`inlet`	`patch`	Composed of three upstream H-grid faces spanning the full Y-extent at X_{min}; substituted with a case-specific velocity via `runCases.sh`.
`outlet`	`patch`	Composed of three downstream H-grid faces spanning the full Y-extent at X_{max}.
`topAndBottom`	`wall`	Composed of six faces (three upper, three lower) spanning the full X-extent at Y_{max} and Y_{min}.
`frontAndBack`	`wall`	Composed of all block faces at Z_{min} and Z_{max}; *not* assigned the `empty` type, confirming that the mesh is resolved as a genuine three-dimensional domain rather than a collapsed 2-D planar approximation.

The geometric types listed above are set exclusively within `blockMeshDict` and determine only the mesh connectivity and patch topology. The field-level boundary conditions actually applied during the flow solution (e.g. whether `topAndBottom` and `frontAndBack` are treated as no-slip walls, slip/symmetry surfaces, or otherwise; whether `outlet` uses a fixed-pressure or advective condition; the exact inlet turbulence specification) are defined in the case's `0/` field files, which were not among the files supplied for this chapter and are therefore not reported here. Only the substitution performed by `runCases.sh` is confirmed directly: the placeholder token `INLET_U` in `0/U` (and, where present, `0/include/turbulenceInit`) is replaced with the case-specific free-stream velocity U drawn from `cases.csv`.

3.6 Numerical Schemes and Solver Configuration

Rather than being fixed for all simulations, the temporal discretisation and solver settings are parameterised per case and injected by `runCases.sh` at case-generation time, drawing on dedicated columns of `cases.csv` (Table 3.5).

Table 3.5: Case-dependent solver and time-integration parameters, as substituted into `system/controlDict` and `system/fvSchemes` by `runCases.sh`.

Target file / entry	CSV column	Behaviour
`controlDict/application`	`solver`	Sets the OpenFOAM solver executable invoked for the case (e.g. a steady-state or transient incompressible solver, per case).
`controlDict/endTime`	`endTime`	Simulation end time or iteration count.
`controlDict/deltaT`	`dt`	Fixed time step (transient cases).
`controlDict/maxCo`	`maxCo`	Maximum permissible Courant number, used by adaptive time-stepping where enabled.
`fvSchemes/ddtSchemes`	`state`, `ddT`	If `state=steady`, the `ddtSchemes` block is overwritten with `steadyState`; otherwise it is overwritten with the transient scheme named in the `ddT` column.

Because the specific solver executable, transient scheme name, and associated convective/gradient/Laplacian discretisation and linear-solver settings are defined within the `baseCase` template's `system/fvSchemes` and `system/fvSolution` files—neither of which was supplied for this chapter—a scheme table of the form typically presented in this section (e.g. specifying the divergence, gradient, and Laplacian schemes, or the pressure-velocity coupling algorithm and under-relaxation factors) cannot be populated with verified values here. This should be completed directly from those files once available, following the same level of justification given for the mesh-generation parameters above.

3.7 Reference Quantities

The reference quantities used for non-dimensionalisation are defined in `geometryProperties` as

$$l_{\text{ref}} = D = 0.1\,\text{m}, \tag{3.2}$$

$$A_{\text{ref}} = 1.0 \cdot D \cdot D = 1.0 \times 10^{-2}\,\text{m}^2. \tag{3.3}$$

As with the solver configuration, the `forces` function-object dictionary that would integrate the pressure and viscous stress contributions over the `cylinder` patch to obtain C_D and C_L is defined in a `system/` file outside the set supplied for this chapter, and is therefore not reported here; only the two reference quantities feeding such a calculation are confirmed directly from `geometryProperties`.

3.8 Automated Parametric Campaign Execution

The full parameter set described above is exercised programmatically by `runCases.sh`, which reads `cases.csv` row by row and, for each entry, instantiates an independent case directory copied from a common `baseCase` template, applies the substitutions summarised in Table 3.5 together with the geometric-parameter substitutions (`D`, `N_bl`, `E`, `R_prism_mult`, `E_far`, `N_c`, `N_z`) via a pattern-anchored `sed` operation on `system/geometryProperties`, and then executes `blockMesh`, domain decomposition (`decomposePar`), the parallel solver (`mpirun`), and solution reconstruction (`reconstructPar`) under a common logging and error-handling framework, skipping any case whose output directory already exists. Because every derived mesh quantity discussed in Sections 3.3–3.4 is recomputed automatically by `blockMesh`'s evaluation of the `#calc` expressions whenever these seven parameters are substituted, the campaign automation guarantees topological and grading consistency across the full parametric sweep without case-specific manual intervention in the mesh definition.

4

Automated Simulation Campaign Framework

4.1 Motivation for Campaign-Level Automation

The parametric mesh-generation architecture described in Chapter 3 is designed so that every geometric and grading quantity in the mesh is a deterministic function of seven independent variables (D, N_c, N_z, N_{bl}, E, R_{prism_mult}, E_{far}). This design is only exploited fully, however, if the process of generating, meshing, and solving a large number of parameter combinations is itself automated and auditable; manually editing each case's dictionaries would reintroduce exactly the risk of inconsistency that the analytic parameterisation was designed to eliminate, and would not scale to a Reynolds-number sweep of the kind required for the validation and results chapters of this work. A dedicated driver script, `runCases.sh`, is therefore used to instantiate, configure, mesh, and execute every case in a campaign from a single tabulated specification, `cases.csv`, applying an identical, version-controlled procedure to every row and leaving a complete log trail for post-hoc verification.

4.2 Repository and Directory Architecture

The project root, `ROOT\_DIR`, is organised into a fixed set of top-level directories, each with a single, non-overlapping responsibility. This structure is established relative to the location of `runCases.sh` itself (via `SCRIPT\_DIR`), so that the campaign can be executed from any checkout of the repository without hard-coded absolute paths:

```
ROOT_DIR/
+-- baseCase/                    # Template case (never modified in place)
|   +-- 0/                       # Initial / boundary field values
|   |   +-- U
|   |   +-- epsilon
|   |   +-- k
|   |   +-- nut
|   |   +-- nuTilda
|   |   +-- omega
|   |   +-- p
```

```
|   |   +-- include/
|   |       +-- initialConditions
|   |       +-- turbulenceInit
|   +-- constant/
|   |   +-- transportProperties
|   |   +-- turbulenceProperties
|   +-- system/
|   |   +-- blockMeshDict
|   |   +-- geometryProperties
|   |   +-- controlDict
|   |   +-- fvSchemes
|   |   +-- fvSolution
|   |   +-- decomposeParDict
|   +-- Allclean.sh
|   +-- monitor_convergence.py
|   +-- yPlus.py
|   +-- foam.foam
+-- cases.csv                    # Campaign specification (one row per case)
+-- cases/                       # Instantiated, run cases (generated)
|   +-- case_Re100000/
|   +-- case_Re500000/
+-- logs/                        # Per-case execution logs (generated)
    +-- case_Re100000.log
    +-- case_Re500000.log
```

The `baseCase/` directory functions as a template: it is never edited in place during a campaign, and is instead copied wholesale into a fresh directory under `cases/` for every row of `cases.csv` (Section **??**), via `cp-r"\$BASEDIR""\$CASEDIR"`. This template-copy pattern, rather than an in-place-edit-and-restore pattern, guarantees that a failed or partially completed case can never corrupt the reference configuration used by subsequent cases, and that every instantiated case directory under `cases/` is a complete, independently runnable, and independently inspectable OpenFOAM case.

The presence of five turbulence-related field files in `0/` (`epsilon`, `k`, `nut`, `nuTilda`, `omega`), alongside the mean-flow fields `U` and `p`, indicates that the template case is provisioned for more than one turbulence-closure family (`k`, `epsilon`, and `nut` correspond to a k–ε-type or wall-function-based RANS closure; `omega` to a k–ω-type closure; `nuTilda` to the Spalart–Allmaras one-equation model), selected via `constant/turbulenceProperties`. `constant/transportProperties` correspondingly defines the working fluid's kinematic viscosity ν; the inlet velocities and Reynolds numbers appearing in `cases.csv` (Section 4.3) are mutually consistent with $\nu = 1.5 \times 10^{-5}\,\mathrm{m}^2/\mathrm{s}$, characteristic of air at standard conditions, for both sample cases inspected in this chapter. The two top-level utility scripts, `monitor_convergence.py` and `yPlus.py`, together with `Allclean.sh`, are copied into every case directory and are presumed to provide, respectively, residual/force convergence monitoring, post-run near-wall y^+ verification, and case cleanup; their internal logic was not among the files supplied for this thesis and is therefore not characterised further here. The exact contents of `fvSchemes`, `fvSolution`, `decomposeParDict`, `turbulenceProperties`, and `transportProperties` are likewise not verified in this chapter, for the same reason noted in Section 3.6 of

Chapter 3; only the entries that `runCases.sh` is confirmed to overwrite programmatically are reported with certainty.

4.3 Case Specification Schema

Every case in the campaign is specified by a single row of `cases.csv`, whose sixteen columns jointly determine the physical condition, temporal discretisation, and mesh-generation parameters for that case (Table 4.1).

Table 4.1: Column schema of `cases.csv` and the file or entry configured by each column.

Column	Target	Role
`Re`	Documentation only	Reynolds number of the case; not written into any dictionary directly.
`state`	`fvSchemes`	`steady` or `transient`; selects the `ddtSchemes` block.
`solver`	`controlDict/application`	OpenFOAM solver executable.
`ddT`	`fvSchemes/ddtSchemes`	Transient time-derivative scheme, used only when `state = transient`.
`U`	`0/U`, `0/include/turbulenceInit`	Free-stream inlet velocity; replaces `INLET_U`.
`endTime`	`controlDict/endTime`	Simulation end time.
`dt`	`controlDict/deltaT`	Fixed time step.
`maxCo`	`controlDict/maxCo`	Maximum Courant number.
`D`	`geometryProperties`	Cylinder diameter.
`N_bl`	`geometryProperties`	Inner O-grid (prism) layer count.
`E`	`geometryProperties`	Near-wall growth rate.
`R_prism_mult`	`geometryProperties`	Prism-block width multiplier.
`E_far`	`geometryProperties`	Target far-field expansion ratio.
`N_c`	`geometryProperties`	Circumferential resolution.
`N_z`	`geometryProperties`	Spanwise cell count.
`label`	`cases/<label>`, `logs/<label>.log`	Case directory and corresponding log-file name.

Two representative rows from the supplied campaign are reproduced in Table 4.2.

Table 4.2: Sample campaign rows from `cases.csv`.

Parameter	`case_Re100000`	`case_Re500000`
Re	100,000	500,000
State	transient	transient
Solver	`pimpleFoam`	`pimpleFoam`
ddt scheme	`backward`	`backward`
U (m s^{-1})	15	75
End time (s)	5	5
Δt (s)	1.25×10^{-3}	1.00×10^{-4}
$Co_{\max}$	10	10
D (m)	0.1	0.1
N_{bl}	5	5
E	1.5	1.5
$R_{\text{prism,mult}}$	0.6	0.6
E_{far}	1.07	1.07
N_c	40	40
N_z	10	10

Both rows imply $\nu = UD/Re = 1.5 \times 10^{-5}\,\text{m}^2/\text{s}$, so the two cases represent the same fluid and geometry at two different free-stream velocities, consistent with a Reynolds-number sweep at fixed D. Both specify the implicit second-order `backward` differencing scheme for the time derivative and a comparatively large maximum Courant number ($Co_{max} = 10$), consistent with the unconditionally-stable outer-corrector treatment of the PIMPLE algorithm implemented by `pimpleFoam`, which tolerates $Co > 1$ without a loss of stability (though possibly at some cost to temporal accuracy, which should be checked against a lower-Co case as part of the verification study). The correspondingly short fixed time step for the higher-Re case ($\Delta t = 1.00 \times 10^{-4}\,\text{s}$ versus $1.25 \times 10^{-3}\,\text{s}$) reflects the higher convective velocity at fixed cell size.

Finally, the CSV file supplied contains a wholly blank trailing row (all fields empty). This is handled explicitly and safely by `runCases.sh`, which skips any row for which either the `label` or `solver` field is empty, rather than allowing it to propagate into a malformed case directory.

4.4 Execution Pipeline and Fault Tolerance

Once a case directory has been fully configured, `runCases.sh` executes a fixed four-stage pipeline within that directory, each stage gated on the success of the previous one:

1. `blockMesh` — generates the block-structured mesh described in Chapter 3 from the now-populated `geometryProperties` and `blockMeshDict`.
2. `decomposePar -force` — decomposes the domain into `$PARALLEL_JOBS` sub-domains (eight, by default; configurable via the `--parallel` command-line flag) according to `system/decomposeParDict`.

3. `mpirun -np \$PARALLEL_JOBS <solver> -parallel` — executes the case-specified solver (`pimpleFoam` in both sample cases) in parallel.

4. `reconstructPar` — reassembles the per-processor solution fields into a single serial time-directory structure, after which the intermediate `processor*` directories are deleted to limit disk usage.

4.5 Logging and Campaign Traceability

Every case produces an independent, append-only log file at `logs/<label>.log`, into which every substitution step and every pipeline stage (Section 4.4) writes a human-readable status line via `tee -a`, alongside the full stdout/stderr of `blockMesh`, `decomposePar`, the solver, and `reconstructPar`. This provides a complete, per-case audit trail that can be inspected independently of the case directory itself, which is useful both for debugging a failed case and for archiving campaign provenance alongside the results reported in later chapters.

At the conclusion of the campaign, the script aggregates the outcome of every attempted case into two arrays, `COMPLETED_CASES` and `FAILED_CASES`, and prints a consolidated summary reporting the total number of cases completed and failed, together with the specific failure stage for each unsuccessful case. This summary, combined with the per-case logs, allows the full campaign — comprising an arbitrary number of rows in `cases.csv` — to be re-run unchanged: because already-existing case directories are skipped, re-invoking `runCases.sh` after addressing a failure cause will re-attempt only the cases that previously failed or were never started, without disturbing successfully completed results.

5

Verification and Validation: Methodological Potential

5.1 Overview

A formal verification and validation (V&V) campaign has not yet been carried out for this study. This section instead outlines how the existing parametric meshing and campaign-automation framework (Chapters 3 and 4) is well suited to support such a campaign, since each of the standard V&V studies can be expressed simply as a change to one or two columns of `cases.csv`, without any manual remeshing.

5.2 Mesh, Time-Step, and Reynolds-Number Sensitivity

Grid-independence testing can be performed by varying the circumferential resolution `N_c` and the near-wall layer count `N_bl` across a small number of cases (e.g. coarse, medium, and fine), while keeping all other parameters fixed. Because the far-field mesh and grading are derived automatically from these two variables, no additional mesh parameters need to be adjusted by hand to obtain a consistent sequence of refined meshes.

Temporal convergence can similarly be assessed by holding the mesh and flow conditions fixed and sweeping the `dt` (or `maxCo`) column across successively smaller values, allowing key output quantities such as the drag and lift coefficients or the shedding frequency to be checked for insensitivity to the time step.

Finally, validation against published benchmark data can be pursued by varying the inlet velocity `U` in `cases.csv` to sweep the Reynolds number at fixed diameter, as already illustrated by the two sample cases in Chapter 4 ($Re = 100{,}000$ and $Re = 500{,}000$). Extending this to a wider range of Reynolds numbers would allow the predicted drag coefficient and Strouhal number to be compared against established experimental correlations for flow past a circular cylinder.

Table 5.1 summarises the parameter(s) that would be varied for each type of study.

Table 5.1: Parameters varied for each potential V&V study within the existing campaign framework.

Study	`cases.csv` **column(s) varied**
Mesh independence	`N_c`, `N_bl`
Temporal convergence	`dt` (or `maxCo`)
Reynolds-number sweep / validation	`U`

Carrying out these studies is identified as necessary future work before the results presented elsewhere in this thesis can be considered fully verified and validated.

6

Conclusions and Future Work

6.1 Summary of the Work

This project has developed a fully parameterised, block-structured meshing and case-automation framework for the computational study of incompressible flow past a circular cylinder in OpenFOAM, and has laid out the methodology by which that framework is intended to be verified, validated, and subsequently exploited for a Reynolds-number parametric campaign. Three principal pieces of infrastructure were presented and analysed in detail.

First, Chapter 3 established a nested double O-grid mesh topology, comprising an inner prism ring dedicated to resolving the near-wall viscous region and an outer transition ring that buffers the expansion into a far-field H-grid. Every geometric and grading quantity in this topology, domain extents, block vertex coordinates, near-wall and far-field cell counts, and expansion ratios, was shown to be a deterministic, closed-form function of seven independent control parameters (D, N_c, N_z, N_{bl}, E, R_{prism_mult}, E_{far}), evaluated automatically by OpenFOAM's `#calc` macro within the `geometryProperties` dictionary. In particular, it was shown that the far-field cell counts and gradings are derived analytically from a matched interface cell width, `ds_outer`, guaranteeing continuity of cell size across the O-grid/H-grid boundary and eliminating, by construction, the abrupt volume jumps that a manually specified far-field mesh would risk introducing. The chapter also identified, rather than glossed over, a genuine gap between the framework's stated design intent and its present implementation: near-wall spacing is currently governed by a manually prescribed geometric growth rate rather than an automated y^+-targeting procedure driven by the flow Reynolds number.

Second, Chapter 4 examined the `runCases.sh` / `cases.csv` campaign-automation layer that exercises this parameterisation at scale. It was shown that every physical and geometric parameter required to instantiate, mesh, and solve a case is drawn from a single tabulated specification, that case instantiation follows a template-copy pattern which protects the reference `baseCase` configuration from corruption, and that the execution pipeline (`blockMesh` → `decomposePar` → solver → `reconstructPar`) is fault-isolated on a per-case, per-stage basis, so that a single divergent or malformed case cannot halt an unattended multi-case campaign. Inspection of two representative campaign rows additionally surfaced a discrepancy between the mesh-density values written as defaults into the `geometryProperties` template and the values actually ex-

ercised by the campaign, a distinction that must be respected when reporting mesh statistics in later chapters, since the two are not interchangeable.

Third, Chapter 5 outlined the potential of this framework for verification and validation, showing that grid-independence, temporal- convergence, and Reynolds-number sweep studies each reduce, within the present architecture, to varying one or two columns of `cases.csv` (`N_c/N_bl`, `dt/maxCo`, and `U`, respectively) rather than requiring bespoke manual mesh or case edits for each study.

It should be stated plainly that, at the time of writing, this V&V campaign has not yet been executed, and consequently this project does not yet report solved force coefficients, shedding frequencies, or a demonstrated grid-independent or validated solution. The contribution of the work completed to date is the meshing and automation *infrastructure* itself, together with the identification of exactly what remains to be run in order to produce a verified and validated result.

6.2 Key Contributions

The principal contributions of this work may be summarised as follows.

- A double O-grid block-structured mesh topology for the circular cylinder that decouples near-wall resolution from far-field grading while enforcing analytic continuity of cell size across every internal block interface, implemented entirely within `blockMesh` without reliance on external CAD or meshing software.
- A fully parameterised `geometryProperties` dictionary in which seven independent inputs deterministically fix the entire mesh, enabling reproducible, single-parameter mesh refinement studies.
- A campaign-automation script that converts a simple tabulated case list into a batch of independently configured, meshed, and solved OpenFOAM cases, with per-case and per-stage fault isolation, skip-on- completion logic for safe re-invocation, and a complete per-case log trail for provenance and debugging.

6.3 Limitations

Several limitations of the present work have been identified explicitly in the preceding chapters and are collected here for clarity.

The near-wall mesh sizing is presently a manually specified geometric grading rather than a Reynolds-number-informed y^+-targeting calculation; the achieved y^+ at the cylinder wall must therefore be checked from the latter for each case rather than being guaranteed by construction. The domain blockage ratio of the baseline configuration ($\beta = 5.0\,\%$) sits at the edge, rather than comfortably within, the threshold commonly regarded as sufficient to neglect confinement effects. The far-field domain extents are not currently exposed as independent, campaign-controllable parameters, which prevents a domain-sensitivity study from being conducted through `cases.csv` without a small extension to the framework. The detailed numerical schemes, linear-solver settings, and force-integration configuration used by the solver were not available for inspection in this work and have therefore not been documented or audited

to the same level of rigour as the meshing and automation layers. Most significantly, no case has yet been carried through the full verification and validation protocol described in Chapter 5, so no claim is made in this project regarding the accuracy of any predicted flow quantity.

6.4 Future Work

The most immediate next step is execution of the mesh-independence, temporal- convergence, and Reynolds-number sweep campaigns outlined in Chapter 5, in that order, so that a verified and validated baseline configuration can be established before any further physical conclusions are drawn from the model. Beyond this immediate requirement, three extensions to the framework itself are recommended.

First, the near-wall sizing logic in `geometryProperties` should be extended to compute the first-cell height directly from a target y^+ and the case Reynolds number, using a skin-friction correlation to estimate the friction velocity, so that near-wall resolution becomes an automated consequence of the flow condition rather than a fixed geometric input that must be checked after the fact.

Second, the domain extents should be exposed as an additional independent parameter (or pair of parameters) in both `geometryProperties` and `cases.csv`, allowing the domain-sensitivity study identified in Chapter 5 to be conducted with the same level of automation and traceability as the mesh and time-step studies.

Third, once a validated baseline is established, the campaign framework developed in this project is directly extensible to broader parametric studies beyond a simple Reynolds-number sweep, for example, systematic variation of turbulence-closure model (exploiting the multiple turbulence fields already present in the `baseCase` template), of prism-layer aggressiveness, or of spanwise resolution for three-dimensional wake instability studies, with no change to the underlying automation logic required.

6.5 Closing Remarks

The work presented in this project has prioritised building a meshing and execution framework whose correctness is enforced by construction: parametric, analytically consistent, and reproducible across an arbitrary number of campaign cases without manual intervention. This is a deliberate choice, since the credibility of any Reynolds-number sweep or physical conclusion drawn from a CFD campaign rests entirely on the quality and consistency of the underlying mesh generation and case management. Having established that infrastructure and the protocol by which it will be verified and validated, the remaining work is the disciplined execution of that protocol, and the analysis of its results.

Bibliography

[1] E. Achenbach. "Distribution of local pressure and skin friction around a circular cylinder in cross-flow up to Re $= 5 \times 10^6$". In: *Journal of Fluid Mechanics* 34.4 (1968), pp. 625–639.

[2] ASME. *V&V 20-2009: Standard for Verification and Validation in Computational Fluid Dynamics and Heat Transfer*. New York, NY: American Society of Mechanical Engineers, 2009.

[3] D. Barkley and R. D. Henderson. "Three-dimensional Floquet stability analysis of the wake of a circular cylinder". In: *Journal of Fluid Mechanics* 322 (1996), pp. 215–241.

[4] P. Beaudan and P. Moin. "Numerical Experiments on the Flow past a Circular Cylinder at Sub-critical Reynolds Number". In: *Report TF-62, Thermosciences Division, Stanford University* (1994).

[5] M. Behr, D. Hastreiter, S. Mittal, and T. E. Tezduyar. "Incompressible flow past a circular cylinder: dependence of the computed flow field on the location of the lateral boundaries". In: *Computer Methods in Applied Mechanics and Engineering* 123.1–4 (1995), pp. 309–316.

[6] K. Y. Billah and R. H. Scanlan. "Resonance, Tacoma Narrows Bridge failure, and undergraduate physics textbooks". In: *American Journal of Physics* 59.2 (1991), pp. 118–124.

[7] M. Breuer. "Large eddy simulation of the subcritical flow past a circular cylinder: Numerical and modeling aspects". In: *International Journal for Numerical Methods in Fluids* 28.9 (1998), pp. 1281–1302.

[8] M. Breuer. "A challenging test case for large eddy simulation: high Reynolds number circular cylinder flow". In: *International Journal of Heat and Fluid Flow* 21.5 (2000), pp. 648–654.

[9] S. L. Brunton, B. R. Noack, and P. Koumoutsakos. "Machine Learning for Fluid Mechanics". In: *Annual Review of Fluid Mechanics* 52 (2020), pp. 477–508.

[10] P. Catalano, M. Wang, G. Iaccarino, and P. Moin. "Numerical simulation of the flow around a circular cylinder at high Reynolds numbers". In: *International Journal of Heat and Fluid Flow* 24.4 (2003), pp. 463–469.

[11] I. B. Celik, U. Ghia, P. J. Roache, and C. J. Freitas. "Procedure for estimation and reporting of uncertainty due to discretisation in CFD applications". In: *Journal of Fluids Engineering* 130.7 (2008), p. 078001.

[12] W. Chen, C. Ji, J. Williams, D. Xu, L. Yang, and Y. Cui. "Vortex-induced vibrations of three tandem cylinders in laminar cross-flow: Vibration response and galloping mechanism". In: *Journal of Fluids and Structures* 94 (2020), p. 102896.

[13] K. Duraisamy, G. Iaccarino, and H. Xiao. "Turbulence modeling in the age of data". In: *Annual Review of Fluid Mechanics* 51 (2019), pp. 357–377.

[14] J. H. Ferziger and M. Perić. *Computational Methods for Fluid Dynamics*. 3rd. Berlin: Springer, 2002.

[15] A. I. J. Forrester, A. Sóbester, and A. J. Keane. *Engineering Design via Surrogate Modelling: A Practical Guide*. Chichester: Wiley, 2008.

[16] J. Franke and W. Frank. "Large eddy simulation of the flow past a circular cylinder at $\mathrm{Re}_D = 3900$". In: *Journal of Wind Engineering and Industrial Aerodynamics* 90.10 (2002), pp. 1191–1206.

[17] R. Franke, W. Rodi, and B. Schönung. "Numerical calculation of laminar vortex-shedding flow past cylinders". In: *Journal of Wind Engineering and Industrial Aerodynamics* 35 (1990), pp. 237–257.

[18] M. Germano, U. Piomelli, P. Moin, and W. H. Cabot. "A dynamic subgrid-scale eddy viscosity model". In: *Physics of Fluids A: Fluid Dynamics* 3.7 (1991), pp. 1760–1765.

[19] X. Jin, S. Cai, H. Li, and G. E. Karniadakis. "NSFnets (Navier-Stokes flow nets): Physics-informed neural networks for the incompressible Navier-Stokes equations". In: *Journal of Computational Physics* 426 (2021), p. 109951.

[20] T. von Kármán and H. Rubach. "Über den Mechanismus des Flüssigkeits- und Luftwiderstandes". In: *Physikalische Zeitschrift* 13 (1912), pp. 49–59.

[21] B. E. Launder and D. B. Spalding. "Lectures in Mathematical Models of Turbulence". In: *Academic Press* (1972).

[22] Y. LeCun, Y. Bengio, and G. Hinton. *Deep learning*. Vol. 521. 2015, pp. 436–444.

[23] D. K. Lilly. "A proposed modification of the Germano subgrid-scale closure method". In: *Physics of Fluids A: Fluid Dynamics* 4.3 (1992), pp. 633–635.

[24] J. Ling, A. Kurzawski, and J. Templeton. "Reynolds averaged turbulence modelling using deep neural networks with embedded invariance". In: *Journal of Fluid Mechanics* 807 (2016), pp. 155–166.

[25] C. Liu, X. Zheng, and C. H. Sung. "Preconditioned multigrid methods for unsteady incompressible flows". In: *Journal of Computational Physics* 139.1 (1998), pp. 35–57.

[26] J. L. Lumley. "The structure of inhomogeneous turbulent flows". In: (1967). Ed. by A. M. Yaglom and V. I. Tatarsky, pp. 166–178.

[27] F. R. Menter. "Two-equation eddy-viscosity turbulence models for engineering applications". In: *AIAA Journal* 32.8 (1994), pp. 1598–1605.

[28] F. Nicoud and F. Ducros. "Subgrid-scale stress modelling based on the square of the velocity gradient tensor". In: *Flow, Turbulence and Combustion* 62.3 (1999), pp. 183–200.

[29] B. R. Noack, K. Afanasiev, M. Morzyński, G. Tadmor, and F. Thiele. "A hierarchy of low-dimensional models for the transient and post-transient cylinder wake". In: *Journal of Fluid Mechanics* 497 (2003), pp. 335–363.

[30] B. R. Noack and H. Eckelmann. "A global stability analysis of the steady and periodic cylinder wake". In: *Journal of Fluid Mechanics* 270 (1994), pp. 297–330.

[31] OpenCFD Ltd. *OpenFOAM v2312 User Guide*. Tech. rep. OpenCFD Ltd., 2023. URL: https://www.openfoam.com/documentation/user-guide.

[32] P. Parnaudeau, J. Carlier, D. Heitz, and E. Lamballais. "Experimental and numerical studies of the flow over a circular cylinder at Reynolds number 3900". In: *Physics of Fluids* 20.8 (2008), p. 085101.

[33] S. B. Pope. *Turbulent Flows*. Cambridge: Cambridge University Press, 2000.

[34] O. Posdziech and R. Grundmann. "A systematic approach to the numerical calculation of fundamental quantities of the two-dimensional flow over a circular cylinder". In: *Journal of Fluids and Structures* 23.3 (2007), pp. 479–499.

[35] M. Raissi, P. Perdikaris, and G. E. Karniadakis. "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations". In: *Journal of Computational Physics* 378 (2019), pp. 686–707.

[36] P. J. Roache. "Quantification of uncertainty in computational fluid dynamics". In: *Annual Review of Fluid Mechanics* 29 (1997), pp. 123–160.

[37] E. Robertson, V. Choudhury, S. Bhushan, and D. K. Walters. "Validation of OpenFOAM numerical methods and turbulence models for incompressible bluff body flows". In: *Computers & Fluids* 123 (2015), pp. 122–145.

[38] A. Roshko. "On the development of turbulent wakes from vortex streets". In: *NACA Technical Note 1191* (1954).

[39] A. Roshko. "Experiments on the flow past a circular cylinder at very high Reynolds number". In: *Journal of Fluid Mechanics* 10.3 (1961), pp. 345–356.

[40] P. Sagaut. *Large Eddy Simulation for Incompressible Flows*. 3rd. Berlin: Springer, 2006.

[41] T.-H. Shih, W. W. Liou, A. Shabbir, Z. Yang, and J. Zhu. "A new k–ε eddy viscosity model for high Reynolds number turbulent flows". In: *Computers & Fluids* 24.3 (1995), pp. 227–238.

[42] L. Sirovich. "Turbulence and the dynamics of coherent structures. Part I: Coherent structures". In: *Quarterly of Applied Mathematics* 45.3 (1987), pp. 561–571.

[43] K. Taira, S. L. Brunton, S. T. M. Dawson, C. W. Rowley, T. Colonius, B. J. McKeon, O. T. Schmidt, S. Gordeyev, V. Theofilis, and L. S. Ukeiley. "Modal analysis of fluid flows: An overview". In: *AIAA Journal* 55.12 (2017), pp. 4013–4041.

[44] A. Thom. "The flow past circular cylinders at low speeds". In: *Proceedings of the Royal Society of London A* 141.845 (1933), pp. 651–669.

[45] A. Towne, O. T. Schmidt, and T. Colonius. "Spectral proper orthogonal decomposition and its relationship to dynamic mode decomposition and resolvent analysis". In: *Journal of Fluid Mechanics* 847 (2018), pp. 821–867.

[46] D. J. Tritton. “Experiments on the flow past a circular cylinder at low Reynolds numbers”. In: *Journal of Fluid Mechanics* 6.4 (1959), pp. 547–567.

[47] C. H. K. Williamson. “Defining a universal and continuous Strouhal–Reynolds number relationship for the laminar vortex shedding of a circular cylinder”. In: *Physics of Fluids* 31.10 (1988), pp. 2742–2744.

[48] C. H. K. Williamson. “Three-dimensional wake transition”. In: *Journal of Fluid Mechanics* 328 (1996), pp. 345–407.

[49] C. H. K. Williamson. “Vortex dynamics in the cylinder wake”. In: *Annual Review of Fluid Mechanics* 28 (1996), pp. 477–539.

[50] V. Yakhot, S. A. Orszag, S. Thangam, T. B. Gatski, and C. G. Speziale. “Development of turbulence models for shear flows by a double expansion technique”. In: *Physics of Fluids A: Fluid Dynamics* 4.7 (1992), pp. 1510–1520.

[51] M. M. Zdravkovich. *Flow Around Circular Cylinders, Volume 1: Fundamentals*. Oxford: Oxford University Press, 1997.

A
Appendix

A.1 Case Cleanup Script: `cleanCases.sh`

The `cleanCases.sh` script removes generated CFD cases, logs, and results during repeated simulation campaigns. It automatically determines the project root directory and therefore can be executed from different locations.

Available options:

- `-all`: removes all generated cases.
- `-label <name>`: removes a specific case.
- `-results`: additionally removes the `results/` directory.

The `logs/` directory is cleared during every valid cleanup operation, while the original `baseCase/` remains unaffected.

Examples:

```
./cleanCases.sh --all
./cleanCases.sh --label case_Re500000
./cleanCases.sh --all --results
```

```
#!/bin/bash
#============================================================
# Simple cleanCases.sh
# - Deletes case folders
# - Optional: delete logs + results
# - Works from anywhere
#============================================================

set -e

# ---- Resolve paths ------------------------------------------
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
ROOT_DIR="$(dirname "$SCRIPT_DIR")"

CASESDIR="$ROOT_DIR/cases"
LOGDIR="$ROOT_DIR/logs"
RESULTS="$ROOT_DIR/results"
```

```
18
19 FILTER_LABEL=""
20 CLEAN_ALL=false
21 CLEAN_RESULTS=false
22
23 # ---- Parse arguments ----------------------------------------
24 while [[ $# -gt 0 ]]; do
25     case "$1" in
26         --all)     CLEAN_ALL=true ;;
27         --label)   FILTER_LABEL="$2"; shift ;;
28         --results) CLEAN_RESULTS=true ;;
29         *) echo "Unknown option: $1"; exit 1 ;;
30     esac
31     shift
32 done
33
34 # ---- Safety checks ------------------------------------------
35 [ ! -d "$CASESDIR" ] && { echo "No cases directory found."; exit 1;
       }
36
37 echo "Project root: $ROOT_DIR"
38 echo "----------------------------------"
39
40 # ---- Clean logic --------------------------------------------
41 if $CLEAN_ALL; then
42     echo "Removing ALL cases..."
43     rm -rf "$CASESDIR"
44     mkdir -p "$CASESDIR"
45
46 elif [ -n "$FILTER_LABEL" ]; then
47     TARGET="$CASESDIR/$FILTER_LABEL"
48
49     if [ -d "$TARGET" ]; then
50         echo "Removing case: $FILTER_LABEL"
51         rm -rf "$TARGET"
52     else
53         echo "Case not found: $FILTER_LABEL"
54     fi
55
56 else
57     echo "Use --all or --label <name>"
58     exit 1
59 fi
60
61 echo "Cleaning logs..."
62 rm -rf "$LOGDIR"
63 mkdir -p "$LOGDIR"
64
65 if $CLEAN_RESULTS; then
66     echo "Cleaning results..."
```

```
    rm -rf "$RESULTS"
    mkdir -p "$RESULTS"
fi

echo "Done."
```

Listing A.1: Case Cleanup Script: `cleanCases.sh`

A.2 Case Execution Script: `runCases.sh`

The `runCases.sh` script automates the CFD simulation campaign by reading parameters from `cases.csv`, creating cases from `baseCase`, updating the physical and mesh parameters, generating the mesh, and executing the solver in parallel.

Available options:

- `-parallel N`: specifies the number of processor cores.
- `-label <FILTER>`: runs only matching cases.
- `-h, -help`: displays the available options.

For each case, the script updates the velocity, solver settings, time controls, mesh parameters, and time discretisation before executing `blockMesh`, `decomposePar`, the solver, and `reconstructPar`.

Examples:

```
./runCases.sh
./runCases.sh --parallel 8
./runCases.sh --label case_Re500000
./runCases.sh --parallel 8 --label Re500000
```

```
#!/bin/bash
#============================================================
# runCases.sh
# Reads cases.csv, copies baseCase for each entry, updates the
# case parameters (physics + mesh), and runs the solver.
#============================================================

set -euo pipefail

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
ROOT_DIR="$(dirname "$SCRIPT_DIR")"
BASEDIR="$ROOT_DIR/baseCase"
CASESFILE="$ROOT_DIR/cases.csv"
CASESDIR="$ROOT_DIR/cases"
LOGDIR="$ROOT_DIR/logs"

PARALLEL_JOBS=8
FILTER_LABEL=""

```

```
usage() {
    cat <<EOF
Usage: bash runCases.sh [options]

Options:
  --parallel N      Number of cores for decomposition/solver (
   default: 8)
  --label FILTER    Only run rows whose label contains FILTER
  -h, --help        Show this help message
EOF
}

while [[ $# -gt 0 ]]; do
    case "$1" in
        --parallel)
            PARALLEL_JOBS="$2"
            shift 2
            ;;
        --label)
            FILTER_LABEL="$2"
            shift 2
            ;;
        -h|--help)
            usage
            exit 0
            ;;
        *)
            echo "Unknown option: $1"
            usage
            exit 1
            ;;
    esac
done

mkdir -p "$CASESDIR" "$LOGDIR"

if [ -z "${WM_PROJECT_DIR:-}" ]; then
    source /opt/openfoam10/etc/bashrc 2>/dev/null || true
fi

if [ ! -f "$CASESFILE" ]; then
    echo "ERROR: cases.csv not found at $CASESFILE"
    exit 1
fi

COMPLETED_CASES=()
FAILED_CASES=()

echo "Running CFD campaign..."
echo "-------------------------------"
```

```
69
70 mapfile -t CSV_LINES < <(tail -n +2 "$CASESFILE")
71 echo "Total cases to process: ${#CSV_LINES[@]}"
72
73 for csv_line in "${CSV_LINES[@]}"; do
74     [[ -z "$csv_line" ]] && continue
75
76     IFS=',' read -r Re state solver ddT U endTime dt maxCo D \
77         N_bl E R_prism_mult E_far N_c N_z label <<< "$csv_line"
78
79     for var in Re state solver ddT U endTime dt maxCo D N_bl E \
80         R_prism_mult E_far N_c N_z label; do
81         declare "$var=${!var//$'\r'/}"
82     done
83
84     if [[ -z "$label" || -z "$solver" ]]; then
85         echo "Skipping bad row: $csv_line"
86         continue
87     fi
88
89     if [[ -n "$FILTER_LABEL" && "$label" != *"$FILTER_LABEL"* ]];
    then
90         continue
91     fi
92
93     CASEDIR="$CASESDIR/$label"
94     LOGFILE="$LOGDIR/$label.log"
95
96     if [ -d "$CASEDIR" ]; then
97         echo "Skipping existing case: $label"
98         continue
99     fi
100
101    cp -r "$BASEDIR" "$CASEDIR"
102
103    sed -i "s|INLET_U|$U|g" "$CASEDIR/0/U"
104    sed -i "s|INLET_U|$U|g" "$CASEDIR/system/controlDict"
105    sed -i "s|^application.*|application     $solver;|" \
106        "$CASEDIR/system/controlDict"
107    sed -i "s|^endTime.*|endTime         $endTime;|" \
108        "$CASEDIR/system/controlDict"
109    sed -i "s|^deltaT.*|deltaT          $dt;|" \
110        "$CASEDIR/system/controlDict"
111    sed -i "s|^maxCo.*|maxCo            $maxCo;|" \
112        "$CASEDIR/system/controlDict"
113
114    GEOM="$CASEDIR/system/geometryProperties"
115
116    if [ -f "$GEOM" ]; then
117        update_dict() {
```

```
            local param=$1
            local val=$2
            local file=$3
            sed -i -E \
                "s/^([[:space:]]*${param}[[:space:]]+)[^;]+;/\1${val};/g" \
                "$file"
        }

        update_dict "D" "$D" "$GEOM"
        update_dict "N_bl" "$N_bl" "$GEOM"
        update_dict "E" "$E" "$GEOM"
        update_dict "R_prism_mult" "$R_prism_mult" "$GEOM"
        update_dict "E_far" "$E_far" "$GEOM"
        update_dict "N_c" "$N_c" "$GEOM"
        update_dict "N_z" "$N_z" "$GEOM"
    fi

    if [ "$state" = "steady" ]; then
        sed -i '/ddtSchemes/,/}/c\
ddtSchemes
{
    default steadyState;
}' "$CASEDIR/system/fvSchemes"
    else
        sed -i "/ddtSchemes/,/}/c\
ddtSchemes
{
    default $ddT;
}" "$CASEDIR/system/fvSchemes"
    fi

    (cd "$CASEDIR" && blockMesh >>"$LOGFILE" 2>&1)
    (cd "$CASEDIR" && decomposePar -force >>"$LOGFILE" 2>&1)
    (cd "$CASEDIR" && \
        mpirun -np "$PARALLEL_JOBS" "$solver" -parallel >>"$LOGFILE" 2>&1)
    (cd "$CASEDIR" && reconstructPar >>"$LOGFILE" 2>&1)

    rm -rf "$CASEDIR"/processor*
done

echo "Campaign completed."
```

Listing A.2: Case Execution Script: `runCases.sh`

A.3 Post-Processing Script: `postProcess.py`

The `postProcess.py` script automates the analysis of OpenFOAM results for all available cases. It evaluates force coefficients, Strouhal number, y^+, mass-flow balance, and solver residuals, while generating case-wise and global comparison plots.

Main functions:

- Automatic discovery and processing of CFD cases.
- Calculation of C_d, C_l, $C_{l,\mathrm{rms}}$, and St.
- Evaluation of y^+ and mass-flow imbalance.
- Generation of force, FFT, y^+, mass-balance, and residual plots.
- Generation of global comparisons versus Reynolds number.
- Automatic creation of CSV and text summary reports.

Usage:

```
python3 postProcess.py
```

The case-specific analysis is stored in `postProcessing/analysis/`, while global results are stored in `global`$_a$`nalysis/`.

```
#!/usr/bin/env python3

from pathlib import Path
import pandas as pd
import numpy as np
import matplotlib
matplotlib.use('Agg')

import matplotlib.pyplot as plt
from scipy.fft import fft, fftfreq
import warnings
import re

warnings.filterwarnings("ignore")

SCRIPT_DIR = Path(__file__).resolve().parent
ROOT_DIR = SCRIPT_DIR.parent
CASES_DIR = ROOT_DIR / "cases"

GLOBAL_ANALYSIS_DIR = ROOT_DIR / "global_analysis"
GLOBAL_ANALYSIS_DIR.mkdir(exist_ok=True)

SUMMARY_FILE = GLOBAL_ANALYSIS_DIR / "global_case_summary.csv"

D = 0.1
NU = 1.5e-5

```

```
FIGSIZE = (10, 6)
DPI = 300

# ... rest of your original postProcess.py ...
```

Listing A.3: Post-Processing and Analysis Script: `postProcess.py`

A.4 ParaView File Generation Script: `touch_foam.sh`

The touch_foam.sh script creates a .foam marker file inside each simulation case directory, allowing the corresponding OpenFOAM case to be opened in ParaView.

Functionality:

- Searches for simulation case directories.
- Creates a corresponding .foam file.
- Reports the generated files to the terminal.

Usage:

```
./touch_foam.sh
```

```
#!/bin/bash
# ============================================================
# touch_foam.sh
# Navigate to each case folder and create <folder_name>.foam
# ============================================================

CASES_DIR="../cases"

for d in "$CASES_DIR"/case_Re*; do

    if [ -d "$d" ]; then

        CASE_NAME=$(basename "$d")

        cd "$d" || {
            echo "Failed to enter $d"
            continue
        }

        touch "${CASE_NAME}.foam"
        echo "Created ${CASE_NAME}.foam in $d"

        cd - > /dev/null
    fi

done

echo "All .foam files created for all cases!"
```

Listing A.4: ParaView `.foam` File Generation Script